Лабораторна робота “Додаткова”

Підоготовлено: Ковалем Олександром

Група; 2ПІ-25Б

Підгупа: 1

Тема: створити графіче вікно використовуючи мову C++. Створити елемети графіки за допомогою мов праограмуванн, обрати спосіб на вибір (обраний - бібліотека C++ “sfml”

Мета: використовуючи наявні занння щоб зробити графічні елементи згідно завданню.

Код програми (зкопійовано з редактору VS Code)

```
#include <SFML/Graphics.hpp>
#include <cmath>
#include <vector>
#include <algorithm>

using namespace std;


struct point { double x, y; };

bool lineIntersection(const point& p, const point& r, const point& q,
const point& s, point& out) {
    double cross_rs = r.x * s.y - r.y * s.x;
    if (cross_rs == 0) return false;
    point qp{q.x - p.x, q.y - p.y};
    double t = (qp.x * s.y - qp.y * s.x) / cross_rs;
    out.x = p.x + r.x * t;
    out.y = p.y + r.y * t;
    return true;
}
```

```
bool segmentIntersection(const point& a, const point& b, const point&
c, const point& d, point& out) {
    point r{b.x - a.x, b.y - a.y};
    point s{d.x - c.x, d.y - c.y};
    double cross_rs = r.x * s.y - r.y * s.x;
    if (cross_rs == 0) return false;
    point ac{c.x - a.x, c.y - a.y};
    double t = (ac.x * s.y - ac.y * s.x) / cross_rs;
    double u = (ac.x * r.y - ac.y * r.x) / cross_rs;
    if (t < -1e-9 || t > 1 + 1e-9 || u < -1e-9 || u > 1 + 1e-9) return
false;
    out.x = a.x + r.x * t;
    out.y = a.y + r.y * t;
    return true;
}

int main() {
    const int width = 500, height = 500;
    sf::RenderWindow window(sf::VideoMode(width, height), "Dodatkove");
    window.setFramerateLimit(60);

    const point center{width / 2.0, height / 2.0};
    const double radius = 220.0;

    vector<point> outer(5);
    double startAngle = -M_PI / 2.0;
    for (int i = 0; i < 5; ++i) {
        double angle = startAngle + i * 2.0 * M_PI / 5.0;
        outer[i].x = center.x + radius * cos(angle);
        outer[i].y = center.y + radius * sin(angle);
    }

    struct vidrizok { point a, b; };
    vector<vidrizok> starSegments;
    for (int i = 0; i < 5; ++i) {
        vidrizok s{ outer[i], outer[(i + 2) % 5] };
        starSegments.push_back(s);
    }

    vector<point> innerPoints;
    for (int i = 0; i < 5; ++i) {
        vidrizok s1 = starSegments[i];
```

```
        vidrizok s2 = starSegments[(i + 1) % 5];
        point ip;
        if (segmentIntersection(s1.a, s1.b, s2.a, s2.b, ip)) {
            innerPoints.push_back(ip);
        }
    }

    point centroid;
    centroid.x = 0;
    centroid.y = 0;
    for (int i = 0; i < innerPoints.size(); i++) {
        centroid.x = centroid.x + innerPoints[i].x;
        centroid.y = centroid.y + innerPoints[i].y;
    }
    centroid.x = centroid.x / innerPoints.size();
    centroid.y = centroid.y / innerPoints.size();

    sf::ConvexShape outerShape;
    outerShape.setPointCount(5);
    for (int i = 0; i < 5; ++i) outerShape.setPoint(i,
sf::Vector2f((float)outer[i].x, (float)outer[i].y));
    outerShape.setFillColor(sf::Color::Transparent);
    outerShape.setOutlineColor(sf::Color::Black);
    outerShape.setOutlineThickness(2.0f);

    vector<sf::VertexArray> starLines;
    for (auto &s : starSegments) {
        sf::VertexArray va(sf::Lines, 2);
        va[0].position = sf::Vector2f((float)s.a.x, (float)s.a.y);
        va[1].position = sf::Vector2f((float)s.b.x, (float)s.b.y);
        va[0].color = sf::Color::Black;
        va[1].color = sf::Color::Black;
        starLines.push_back(va);
    }

    sf::ConvexShape innerShape;
    if (!innerPoints.empty()) {
        innerShape.setPointCount(innerPoints.size());
        for (size_t i = 0; i < innerPoints.size(); ++i) {
            innerShape.setPoint(i,
sf::Vector2f((float)innerPoints[i].x, (float)innerPoints[i].y));
        }
        innerShape.setFillColor(sf::Color(220, 80, 80, 200));
```

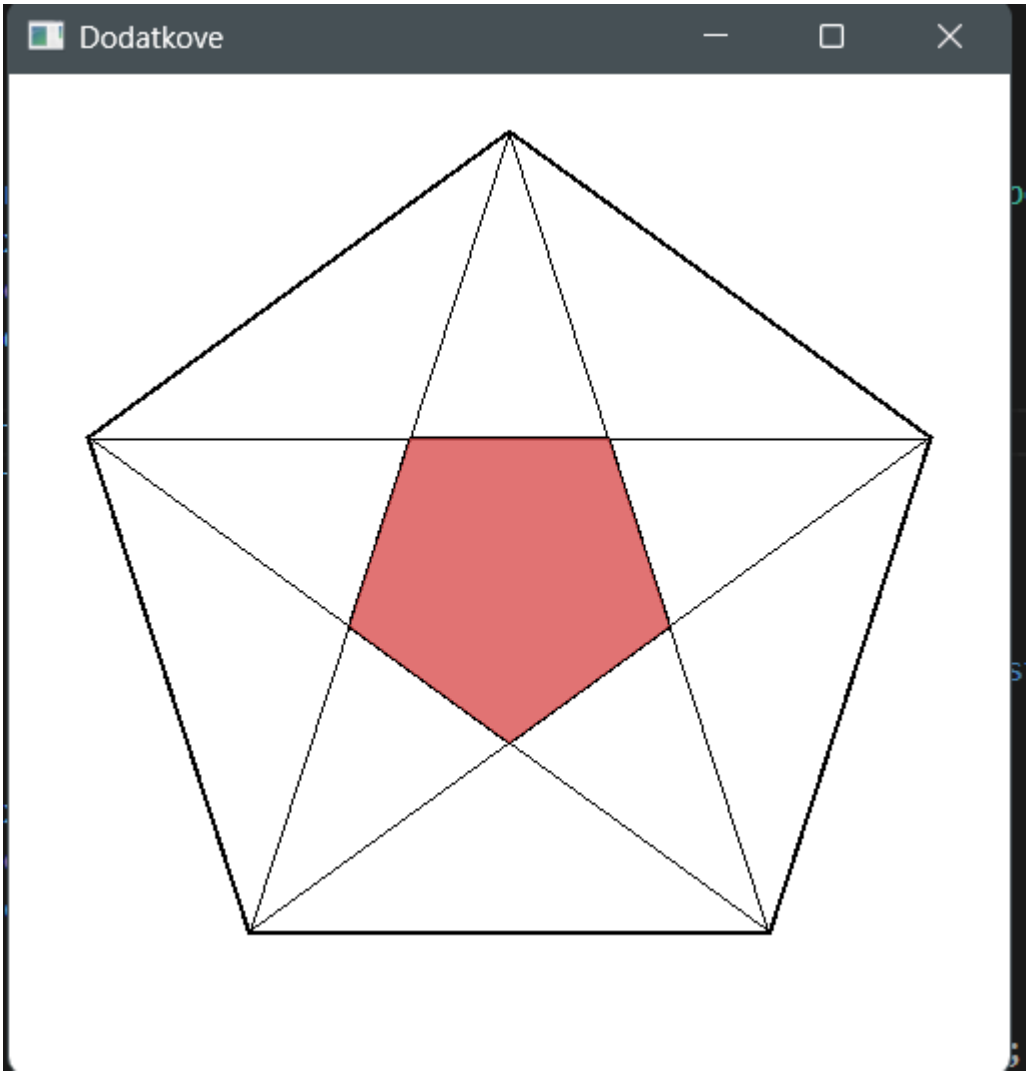
```
        innerShape.setOutlineColor(sf::Color::Black);
        innerShape.setOutlineThickness(1.0f);
    }

    while (window.isOpen()) {
        sf::Event ev;
        while (window.pollEvent(ev)) {
            if (ev.type == sf::Event::Closed) window.close();
        }
        window.clear(sf::Color::White);
        window.draw(outerShape);
        for (auto &va : starLines) window.draw(va); /
        if (innerPoints.size() == 5) window.draw(innerShape);
        window.display();
    }

    return 0;
}
#include <SFML/Graphics.hpp>
#include <cmath>
#include <vector>
#include <algorithm>

using namespace std;


struct point { double x, y; };

bool lineIntersection(const point& p, const point& r, const point& q,
const point& s, point& out) {
    double cross_rs = r.x * s.y - r.y * s.x;
    if (cross_rs == 0) return false;
    point qp{q.x - p.x, q.y - p.y};
    double t = (qp.x * s.y - qp.y * s.x) / cross_rs;
    out.x = p.x + r.x * t;
    out.y = p.y + r.y * t;
    return true;
}

bool segmentIntersection(const point& a, const point& b, const point&
c, const point& d, point& out) {
    point r{b.x - a.x, b.y - a.y};
    point s{d.x - c.x, d.y - c.y};
```

```
    double cross_rs = r.x * s.y - r.y * s.x;
    if (cross_rs == 0) return false;
    point ac{c.x - a.x, c.y - a.y};
    double t = (ac.x * s.y - ac.y * s.x) / cross_rs;
    double u = (ac.x * r.y - ac.y * r.x) / cross_rs;
    if (t < -1e-9 || t > 1 + 1e-9 || u < -1e-9 || u > 1 + 1e-9) return
false;
    out.x = a.x + r.x * t;
    out.y = a.y + r.y * t;
    return true;
}

int main() {
    const int width = 500, height = 500;
    sf::RenderWindow window(sf::VideoMode(width, height), "Dodatkove");
    window.setFramerateLimit(60);

    const point center{width / 2.0, height / 2.0};
    const double radius = 220.0;

    vector<point> outer(5);
    double startAngle = -M_PI / 2.0;
    for (int i = 0; i < 5; ++i) {
        double angle = startAngle + i * 2.0 * M_PI / 5.0;
        outer[i].x = center.x + radius * cos(angle);
        outer[i].y = center.y + radius * sin(angle);
    }

    struct vidrizok { point a, b; };
    vector<vidrizok> starSegments;
    for (int i = 0; i < 5; ++i) {
        vidrizok s{ outer[i], outer[(i + 2) % 5] };
        starSegments.push_back(s);
    }

    vector<point> innerPoints;
    for (int i = 0; i < 5; ++i) {
        vidrizok s1 = starSegments[i];
        vidrizok s2 = starSegments[(i + 1) % 5];
        point ip;
        if (segmentIntersection(s1.a, s1.b, s2.a, s2.b, ip)) {
            innerPoints.push_back(ip);
        }
```

```
    }

    point centroid;
    centroid.x = 0;
    centroid.y = 0;
    for (int i = 0; i < innerPoints.size(); i++) {
        centroid.x = centroid.x + innerPoints[i].x;
        centroid.y = centroid.y + innerPoints[i].y;
    }
    centroid.x = centroid.x / innerPoints.size();
    centroid.y = centroid.y / innerPoints.size();

    sf::ConvexShape outerShape;
    outerShape.setPointCount(5);
    for (int i = 0; i < 5; ++i) outerShape.setPoint(i,
sf::Vector2f((float)outer[i].x, (float)outer[i].y));
    outerShape.setFillColor(sf::Color::Transparent);
    outerShape.setOutlineColor(sf::Color::Black);
    outerShape.setOutlineThickness(2.0f);

    vector<sf::VertexArray> starLines;
    for (auto &s : starSegments) {
        sf::VertexArray va(sf::Lines, 2);
        va[0].position = sf::Vector2f((float)s.a.x, (float)s.a.y);
        va[1].position = sf::Vector2f((float)s.b.x, (float)s.b.y);
        va[0].color = sf::Color::Black;
        va[1].color = sf::Color::Black;
        starLines.push_back(va);
    }

    sf::ConvexShape innerShape;
    if (!innerPoints.empty()) {
        innerShape.setPointCount(innerPoints.size());
        for (size_t i = 0; i < innerPoints.size(); ++i) {
            innerShape.setPoint(i,
sf::Vector2f((float)innerPoints[i].x, (float)innerPoints[i].y));
        }
        innerShape.setFillColor(sf::Color(220, 80, 80, 200));
        innerShape.setOutlineColor(sf::Color::Black);
        innerShape.setOutlineThickness(1.0f);
    }

    while (window.isOpen()) {
```

```
        sf::Event ev;
        while (window.pollEvent(ev)) {
            if (ev.type == sf::Event::Closed) window.close();
        }
        window.clear(sf::Color::White);
        window.draw(outerShape);
        for (auto &va : starLines) window.draw(va); /
        if (innerPoints.size() == 5) window.draw(innerShape);
        window.display();
    }

    return 0;
}
```

Результат програми



Висновок:

Виконуючи цю роботу я навчився використовувати біблоотеку з C++ для створення графічних зображень за допомогою коду. Навчися використовувати математичні формули у вигляді коду для створення зображення.